

Kreativre
Karakana

KARAKANA APP

Prepared for Kreative Karakana

Written
JAN, 2026

Author
JEHOSHAPHAT OBOL

KARAKANA APP DOCUMENTATION

1. INTRODUCTION

This document provides an overview of the **current state of the app** and serves as a guide for future developers who will continue its development. It highlights both the **current limitations** and **planned improvements**, giving context to technical decisions made during the project lifecycle.

The primary goals of this documentation are:

- To provide a **clear understanding of the existing codebase**, including parts generated by FlutterFlow and features added manually.
- To **outline the limitations and technical debt** inherited from initial development choices.
- To offer **recommendations for improvement**, including refactoring or rebuilding strategies, to ensure the app can be maintained and extended reliably in the future.

This document is intended for **developers who will take over or extend the project**, as well as for stakeholders who need a high-level understanding of the app's current condition and potential next steps.

2. PROJECT OVERVIEW

The project is an **Edutech platform** focused on providing **entrepreneurship training** to its users. The app's goal is to deliver educational content, interactive learning experiences, and tools to support aspiring entrepreneurs in building their skills and knowledge.

Project Timeline

- **Initial Development (FlutterFlow Phase, 2 months):** The project was initially developed using **FlutterFlow** to accelerate delivery due to time constraints. This allowed for rapid prototyping and initial feature deployment but introduced technical limitations and reduced maintainability.
- **Manual Development Phase (Next 3 months):** After two months, FlutterFlow was dropped, and additional features were added manually in the codebase. However, due to the artifacts and structure generated by FlutterFlow, extending the app proved difficult, highlighting the **inadequacy of the original implementation**.
- **Current State:** It became clear that the project requires either a **full refactor** or a **complete rebuild** to achieve maintainability and scalability.

High-Level Features

The platform provides:

- Structured entrepreneurship courses and modules
- User registration, login, and profile management
- Ambassadors program

3. ARCHITECTURE & PROJECT STRUCTURE

The app is built using **Flutter**, with the initial version generated through **FlutterFlow** and subsequent features added manually. The current codebase contains a mixture of **FlutterFlow artifacts** and **custom implementations**, which can make extending functionality challenging.

High-Level Structure

```
lib/
├─ flutter_flow/           # Core FlutterFlow-generated
utilities, widgets, navigation, and custom functions
├─ custom_code/           # Manually implemented custom
actions and widgets
├─ actions/               # Shared action utilities
├─ auth/                  # Custom authentication logic and
user providers
├─ backend/               # API calls, Firebase configuration,
local data, and schema definitions
├─ components/            # Reusable widgets and UI components
├─ pages/                 # Screen-level UI organized by
modules (accounts, auth, course, payments, etc.)
├─ webview/               # Webview implementation for
embedded pages
├─ main.dart              # Application entry point
├─ app_state.dart         # Global app state management
├─ app_constants.dart     # Global constants
└─ index.dart             # Central import file
```

FlutterFlow vs Manual Code

- **FlutterFlow-Generated:**

- The `flutter_flow/` folder contains widgets, navigation utilities, form controllers, and helper functions generated by FlutterFlow.
- Initial screens, basic layouts, and default theme configurations were mostly handled here.

- **Manual Additions:**

- `custom_code/` contains manually implemented widgets and actions to

extend functionality beyond FlutterFlow's capabilities (e.g., `mux_player`, custom bottom sheets, profile checks).

- `auth/custom_auth/` implements custom authentication flows outside of FlutterFlow's built-in solutions.
- `backend/` handles API calls, Firebase configurations, and local data utilities.
- Most screens under `pages/` have been manually modified or extended with features not supported by FlutterFlow.

Key Observations & Patterns

- The project follows a **modular page-based structure**, with each major feature or module organized under `pages/`.
- **Reusable components** are kept in `components/`, but many UI widgets are still FlutterFlow-generated, making consistent styling and behavior extensions challenging.
- The global state is managed via `app_state.dart`, but coupling between FlutterFlow-generated code and manual implementations can cause maintenance difficulties.
- Backend integrations and API calls are centralized in `backend/api_requests/`, which provides a single point for network communication.

4. NAVIGATION FLOW

Main Pages (Top-Level Tabs)

The app has **4 main pages** accessible via the bottom navigation bar (or equivalent UI element):

Page	Purpose
Home	Displays feeds, banners, and highlights. Main landing page after login.
Explore	Allows browsing of courses, popular or recommended content.
Notifications	Shows alerts, updates, and system messages for the user.
Accounts	User profile, account settings, linked payment methods, and other personal info.

Authentication Flow

Authentication screens are **outside the main tabs** and redirect the user into `/home` after login/signup:

Route	Purpose
<code>/onboarding</code>	App introduction, first-time user flow.
<code>/login</code>	Login screen for returning users.
<code>/signup</code>	Sign-up screen for new users.
<code>/forgotPassword</code>	Start password reset process.
<code>/emailVerification</code>	Email confirmation flow after sign-up.

Notes:

- Users cannot access the main tabs without authentication.
- `/onboarding` appears only on first launch.

Course Flow

Courses are the core content of the app. Relevant routes include:

Route	Purpose
<code>/courseDetails</code>	Shows course info and "join" options.
<code>/studentLearning</code>	Learning interface for enrolled courses.
<code>/courseReviews</code>	Allows reading and leaving reviews.
<code>/review_form</code> , <code>/review_item</code> , <code>/review_reply_form</code>	Components for adding and replying to reviews.
<code>/enrolledCoursesListView</code>	List of courses a user is enrolled in.
<code>/classroomView</code>	Interactive classroom or lesson view.

Notes:

- Course flows often **branch dynamically**, depending on enrollment and progress.
- Many course-related screens were initially generated via FlutterFlow but now include **custom components**.

Payments Flow

Payment screens integrate course enrollment and wallet management:

Route	Purpose
<code>/paymentPage</code>	Payment interface for joining a course.
<code>/paymentStatus</code>	Shows success/failure of transactions.
<code>/paymentHistory</code>	Historical records of all payments.
<code>/walletForm</code>	Allows topping up the user wallet.
<code>/trainersCheckoutHistory</code>	Historical records for trainer payouts.

Account & Profile Flow

Account screens allow users to manage personal info and trainer applications:

Route	Purpose
<code>/account</code>	Landing page for account-related actions.
<code>/profile</code>	View profile.
<code>/profileForm</code>	Edit user profile.
<code>/mastercardForm</code>	Add or manage payment cards.
<code>/trainersApplicationForm</code>	Application form to become a trainer.

Notes:

- Many account screens include **conditional navigation**, e.g., if a profile is incomplete, the system redirects to `/profileForm`.
- Some forms include **nested steps** represented by custom widgets like `ResponsiveCarousel`.

Other Flows

- `/notifications` → Lists alerts and updates.
- `/recommencationListView`, `/popularListView`, `/weeklyChoiceListView`, `/freeCoursesListView` → Explore filters and content lists.
- `/commingSoon` → Placeholder screen for upcoming features.
- `/webview` → Shows external content inside the app.
- `/customerSupport`, `/messageView` → Support chat and ticket flow.
- `/termsAndConditions` → Legal screen.

Navigation Patterns

1. **GoRouter**: App uses FlutterFlow's GoRouter for all navigation.

2. Nested Navigation:

- Tabs (`Home` , `Explore` , `Notifications` , `Accounts`) maintain independent stacks.
- Some modals/bottom sheets open on top of tab stacks (e.g., review forms, trainer application steps).

3. Dynamic Routes:

- `/courseDetails` and similar pages load based on the selected course ID.
- `/webview` can open any URL.

4. FlutterFlow Artifacts:

- Most navigation logic was scaffolded by FlutterFlow.
- Manual flows now exist for **custom forms, carousels, and conditional redirects**.

5. FEATURES

The application is **fully functional** in its current state. All core modules including **courses, payments, accounts, notifications, and support flows** operate as intended.

Core Features

○ **Courses & Learning**

- Users can browse, enroll, and complete entrepreneurship courses.
- Learning interfaces, including classroom views, lessons, and progress tracking, are fully implemented.
- Course reviews and student feedback mechanisms are operational.

○ **Payments & Wallet Management**

- Payment pages, wallet top-ups, and historical transaction views are functional.
- Trainer payouts and checkout history features are available.
- **Accounts & Profiles**
 - Users can manage personal information, profile updates, and linked payment methods.
 - Trainer application forms and account settings are fully operational.
- **Notifications & Communications**
 - Alerts, messages, and support ticket flows function as expected.

Implementation Notes

- Most of the **manual code** was written following **FlutterFlow's structure and naming conventions**, which are often verbose, cryptic, and difficult to maintain.
- There is **no new architectural pattern or optimization** in the manual code; it primarily mirrors the FlutterFlow scaffold.
- While features are working, **extending or modifying them is challenging** due to the inherited FlutterFlow artifacts and code style.

Limitations & Considerations

- Adding new functionality or refactoring existing modules is non-trivial because the **code structure is tightly coupled to FlutterFlow-generated patterns**.
- Future developers should be aware that **most files and widgets follow FlutterFlow's verbose and sometimes confusing conventions**, and changes may require significant effort to maintain readability and stability.

6. DEPENDENCIES & INTEGRATIONS

The app relies on a combination of **Flutter packages**, **custom backend services**, and third-party integrations to provide its functionality.

1. Authentication

- **Google Sign-In**: Users can log in or sign up using their Google accounts.
- **Custom Authentication**: Additional authentication flows are implemented manually, including email/password and profile management.

2. Backend Services

- **Custom REST API Endpoints**: All app data and business logic are powered by **REST APIs built by Kreative Karakana**.
 - Courses, payments, user profiles, and communications are handled via these endpoints.
 - API calls are centralized in `lib/backend/api_requests/`.

3. Notifications

- **Firestore Cloud Messaging (FCM)**: Push notifications are used to deliver alerts, updates, and messages to users.
 - Configured via `lib/backend/firebase/firebase_config.dart`.
 - Supports both system notifications and in-app messaging.

4. Other Dependencies

- Standard **Flutter packages** (e.g., for UI components, state management, and networking) are included via `pubspec.yaml`.
- **Custom widgets** (like Mux video player, Quill viewer) extend FlutterFlow defaults where required.

Notes

- Most integrations are **fully functional**, but extending them may require

understanding both the FlutterFlow-generated scaffolding and the manually added custom code.

- Any future refactor should review **API calls, authentication flows, and FCM integration** to ensure maintainability and scalability.

7. KNOWN ISSUES & TECHNICAL DEBT

The current codebase reflects the **inherent limitations of the initial FlutterFlow implementation**, compounded by subsequent manual additions. While the app is fully functional, it carries significant **technical debt** that future developers should be aware of.

Main Pain Points

1. Maintainability

- The code is **difficult to read and understand** due to FlutterFlow's verbose and cryptic naming conventions.
- Most manual code mimics FlutterFlow's style, so there are **few clear patterns or clean abstractions**.

2. Scalability

- Extending the app with new features is non-trivial because the **FlutterFlow-generated structure is tightly coupled** with existing logic.
- Adding complex functionality may require substantial refactoring of the existing code.

3. Code Structure

- Files and folders follow **FlutterFlow scaffolding**, resulting in **deeply nested directories** and **repetitive code**.
- Components, actions, and widgets often lack clear separation of concerns.

4. Readability

- Many function and widget names are unintuitive, making it hard to track behavior or debug issues.
- Documentation within the code is minimal.

Limitations of Current Features

- Features work as intended but are **fragile**: small changes can introduce unexpected bugs.
- Conditional navigation, forms, and dynamic UI elements are especially **tightly coupled** with FlutterFlow artifacts.
- Future enhancements may require **partial or complete refactor** to improve maintainability and reduce risk.

Summary

The technical debt in this project means that, while the app is usable, **continuing development without refactoring or rebuilding is challenging**. Future developers should consider **a clean rebuild or significant refactor** to ensure the platform can scale effectively.

8. RECOMMENDATIONS & NEXT STEPS

Given the current state of the app, continuing development requires careful consideration. While all features are functional, the **FlutterFlow-generated structure and resulting technical debt** make maintenance and extension challenging.

Option 1: Refactor

- A **gradual refactor** is possible but requires an initial investment of time:
 - Developers must **thoroughly understand the existing codebase**, including FlutterFlow artifacts and manually added features.
 - Refactoring can then proceed module by module, improving readability, maintainability, and scalability over time.
- Pros: Preserves existing features and data structures.
- Cons: Initial phase may be steep and complex; risk of introducing bugs if the structure is not fully understood.

Option 2: Rebuild from Scratch

- A **full rebuild** offers a fresh start, allowing developers to:
 - Implement **clean architecture patterns** from the outset.
 - Avoid inherited FlutterFlow artifacts and confusing conventions.
 - Potentially achieve a **quicker and cleaner implementation** of new features.
- Pros: Cleaner code, scalable architecture, easier long-term maintenance.
- Cons: Requires reproducing existing features; some initial setup and testing required.

Recommended Approach

- **Evaluate the team's capacity** and project priorities.
- For **short-term speed**, a rebuild may provide a faster, cleaner start.
- For **incremental improvement** and preserving existing workflows, a refactor may be pursued if the team can dedicate time to understanding the current codebase.
- Regardless of the approach, all future development should:
 - Follow **clear coding conventions**.
 - Maintain **modular and readable architecture**.

- Minimize tight coupling between components.

Lessons Learned

- Using **FlutterFlow accelerated initial delivery** but introduced long-term maintainability challenges.
- Future development should prioritize **clean architecture, modular design, and thorough documentation** to avoid accumulating technical debt.